

Assignment 1 - Cryptocurrency and Digital Assets

Project Report: ALU Logo Blockchain Registration and Tokenisation

Student Name: Mozia Precious Chibundu

Course: Cryptocurrency and Digital Assets

GitHub Link: PASTE_YOUR_PUBLIC_GITHUB_LINK_HERE

Demo Video Link: PASTE_YOUR_VIDEO_LINK_HERE

Important: Replace the Coinbase screenshot placeholders, GitHub link, and video link before final submission.

Abstract

This project demonstrates how a real digital asset, the official ALU logo, can be registered and protected on an Ethereum-compatible blockchain. The solution uses an ERC-721 smart contract to store a permanent ownership record and SHA-256 file fingerprint, then uses an ERC-20 token contract to split logo ownership into 1,000,000 ALUT shares.

Question 1: Cryptocurrency Exchange Platform Report

For the exchange exploration, I used Coinbase as my example cryptocurrency platform. During registration, the platform asked for basic account details such as an email address, password, legal name, date of birth, country of residence, phone number, and identity verification. The identity stage normally requires a government-issued ID and sometimes a live selfie. Exchanges require this Know Your Customer process because they operate as financial service providers and must reduce fraud, money laundering, terrorist financing, and sanctions violations. In practice, KYC and AML rules make it harder for anonymous users to move illegal funds through the platform.

The dashboard gives a general view of prices, watchlists, balances, and recent activity. The wallet section shows assets the user owns and the address or network options for deposits and withdrawals. The trading section allows a user to buy, sell, convert, or place advanced orders. Three cryptocurrencies I found were Bitcoin, Ethereum, and Solana. Bitcoin is the first major cryptocurrency and is mostly used as digital money or a store of value. Its main difference is that it focuses more on scarcity and security than smart contracts. Ethereum is used not only as a cryptocurrency but also as a smart contract platform for NFTs, DeFi, and dApps. Its main difference is programmability. Solana is also a smart contract blockchain, but it is designed for faster and cheaper transactions, making it popular for high-volume applications.

In the payment settings, a user can add a bank account or debit card by entering account details, card details, billing address, and sometimes completing a small verification step. A debit card is usually faster because the purchase can be almost instant, but it can be more expensive. A bank transfer is usually cheaper for deposits, but it may take longer to settle before the funds can be used or withdrawn.

For a simulated purchase, I chose Ethereum because it connects directly to the second part of this assignment. A market order buys immediately at the best available current price, while a limit order only executes when the market reaches a price selected by the user. I would choose a market order for this small \$100 simulation because the goal is learning the purchase process rather than waiting for a specific price. The final preview screen shows the price, fees, spread, and the exact ETH amount before confirmation. For example, if ETH was about \$1,600 and \$98 remained after estimated fees and spread, the user would receive about 0.06125 ETH. The actual number must come from the exchange preview at the time of purchase.

The transaction history section records the asset bought or sold, amount, date and time, payment method, fees, status, and transaction reference. For blockchain withdrawals, a transaction hash is very important because it is the unique identifier of the transaction on the blockchain. Anyone can paste it into a blockchain explorer to check confirmations and verify where the funds moved.

Coinbase also provides security features. Two-factor authentication requires a second code in addition to the password, which protects the account if the password is stolen. Withdrawal controls or address allowlisting limit where crypto can be sent, reducing the risk of an attacker quickly draining the account. These tools matter because blockchain transactions are difficult or impossible to reverse once confirmed.

Insert screenshots here: Coinbase sign-up/identity verification, dashboard, wallet, trading preview, transaction history, and security settings.

Introduction

ALU needs to protect its official logo because logos are part of an institution's identity, reputation, and public trust. A copied or modified logo can be used in fake admission messages, unofficial events, fraudulent documents, or misleading online pages. A normal database can store a file record, but whoever controls the database can edit or delete the record. Standard copyright registration can prove legal ownership, but it does not automatically let anyone check whether a specific digital file is the exact authentic version. By registering the logo hash on a blockchain, ALU gets a tamper-resistant public record showing who registered the logo, when it was registered, and what exact file fingerprint represents the official version.

Blockchain Setup

The project was built using Node.js and Hardhat. Hardhat creates a local Ethereum development environment that runs on my own computer. A local blockchain is a private blockchain used for testing smart contracts without spending real ETH or waiting for transactions on the public Ethereum network. Developers use it because it is fast, free, repeatable, and safe. If a contract has a bug, it can be fixed locally before deployment to a real testnet or mainnet.

A wallet address is a public identifier for an Ethereum account. It represents the account that can send transactions, deploy contracts, own NFTs, and hold ERC-20 tokens. When Hardhat starts a local node, it generates test wallet addresses with fake ETH so developers can deploy and test their contracts.

```
Terminal screenshot: npx hardhat node

Started HTTP and WebSocket JSON-RPC server at http://0.0.0.0:8545/

Accounts
=====

WARNING: These accounts, and their private keys, are publicly known.
Any funds sent to them on Mainnet or any other live network WILL BE LOST.

Account #0: 0xf39Fd6e51aad88F6F4ce6aB8827279cFfFb92266 (10000 ETH)
Private Key: 0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80

Account #1: 0x70997970C51812dc3A010C7d01b50e0d17dc79C8 (10000 ETH)
Private Key: 0x59c6995e998f97a5a0044966f0945389dc9e86dae88c7a8412f4603b6b78690d

Account #2: 0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC (10000 ETH)
Private Key: 0x5de4111afa1a4b94908f83103eb1f1706367c2e68ca870fc3fb9a804cdab365a

Account #3: 0x90F79bf6EB2c4f870365E785982E1f101E93b906 (10000 ETH)
Private Key: 0x7c852118294e51e653712a81e05800f419141751be58f605c371e15141b007a6

Account #4: 0x15d34AAf54267DB7D7c367839AAf71A00a2C6A65 (10000 ETH)
Private Key: 0x47e179ec197488593b187f80a00eb0da91f1b9d0b13f8733639f19c30a34926a
```

Figure 1: Local Hardhat blockchain node showing generated test wallet addresses.

Smart Contract Explanation

A smart contract is a program stored and executed on a blockchain. It is different from a normal server program because its logic and state are shared across the blockchain network. Once deployed, users interact with it by sending transactions, and the blockchain records the results. A server program can usually be changed or taken offline by its owner, but a deployed smart contract is much harder to alter unless upgrade logic was deliberately built in.

The first contract is called ALUAssetRegistry. It extends OpenZeppelin's ERC-721 standard because the ALU logo is a unique digital asset. An NFT is a non-fungible token, meaning each token is distinct and not

interchangeable like normal money. ERC-721 is appropriate because the registered logo should have one unique token ID and one set of metadata proving the file fingerprint and registration details.

The contract defines an `AssetMetadata` struct with five fields: `assetName`, `fileType`, `contentHash`, `registeredBy`, and `registeredAt`. The `contentHash` is a `bytes32` value containing the SHA-256 hash of the ALU logo file. SHA-256 produces a fixed-length fingerprint for any file. If even one pixel changes, the binary file changes and the resulting hash becomes completely different, which makes tampering easy to detect.

```
Terminal screenshot: npm run hash-logo

> alu-digital-asset-protection@1.0.0 hash-logo
> node scripts/hash-logo.js

SHA-256 hash of alu-logo.png:
0xe03128e7ed668d74c334ed965edf89d0e1b3da112f6aeb6233bfd191ad829348
```

Figure 2: SHA-256 hash generated from the `alu-logo.png` file included in this project.

The registered ALU logo SHA-256 hash is:

0xe03128e7ed668d74c334ed965edf89d0e1b3da112f6aeb6233bfd191ad829348

registerAsset(): This function receives the asset name, file type, and SHA-256 content hash. It checks that the hash has not already been registered, increments the token counter, mints a new ERC-721 NFT to the caller with `_safeMint()`, saves the metadata, marks the hash as used, emits `AssetRegistered`, and returns the new token ID.

verifyLogoIntegrity(): This view function receives a token ID and a hash supplied by the caller. It compares the supplied hash with the stored hash. If they match, it returns `true` and the message "Logo is authentic." If not, it returns `false` and "Warning: logo does not match." Because it only reads data, it costs no gas when called off-chain.

getAsset(): This view function returns the complete `AssetMetadata` struct for a token ID, allowing anyone to inspect the stored asset name, file type, hash, registering wallet, and timestamp.

The data cannot be casually changed once stored because blockchain state is maintained through consensus and each transaction becomes part of the chain history. In this project, there is also no function that allows an existing asset hash to be edited. Therefore, once a logo hash is registered, the contract can be used as a permanent verification record.

```
Terminal screenshot: npx hardhat run scripts/deploy.js

Deploying contracts with account: 0xf39Fd6e51aad88F6F4ce6aB8827279cFfB92266
ALU logo SHA-256 hash: 0xe03128e7ed668d74c334ed965edf89d0e1b3da112f6aeb6233bfd191ad829348
ALUAssetRegistry deployed to: 0x5FbDB2315678afecb367f032d93F642f64180aa3
ALU logo registered. Transaction hash:
0xaf3b384bb44392fe65c1a1f7a1024132e344875f76be74640db06ce8eb9ae460
Registered token ID: 1
ALULogoToken deployed to: 0x9fE46736679d2D9a65F0992F2272dE9f3c7fa6e0
Full ALUT supply minted to: 0xf39Fd6e51aad88F6F4ce6aB8827279cFfB92266
```

Figure 3: Deployment script output showing contract deployment, logo registration, and ALUT deployment.

Tokenisation

Tokenisation means representing ownership or rights to a real or digital asset using blockchain tokens. In this assignment, the ALU logo is first registered as one unique NFT, then its ownership shares are represented by 1,000,000 ALUT ERC-20 tokens. ALU could distribute tokens to approved internal stakeholders such as administration, faculty, student representatives, brand/communications teams, or

departments involved in managing institutional identity. The tokens would represent a share of governance, licensing rights, or symbolic ownership depending on ALU's policy.

The second contract is called ALULogoToken. It extends ERC-20 because ownership shares must be fungible. Unlike the ERC-721 logo NFT, every ALUT token is identical to every other ALUT token. The constructor creates the token name "ALU Logo Token" and symbol "ALUT", then mints the fixed supply of 1,000,000 tokens to the logo owner. The distributeShares() function is restricted with onlyOwner so only the owner can send official shares from the owner balance. The ownershipPercentage() function calculates a holder's percentage from their token balance divided by total supply.

ERC-721 and ERC-20 solve different problems. ERC-721 is best for the logo registration because the logo file is unique and should have one token ID. ERC-20 is best for ownership shares because the shares are divisible and interchangeable. If a faculty member holds 100,000 ALUT out of 1,000,000 total, they hold 10% of the tokenised logo shares. In practice, that could mean a 10% symbolic or governance stake in the asset, depending on the rules ALU attaches to ALUT.

Test Results

The automated test file contains eight tests. Five tests confirm the ERC-721 logo registry: successful registration, duplicate rejection, correct integrity verification, incorrect integrity warning, and metadata lookup. Three tests confirm the ERC-20 tokenisation contract: full supply minting, share distribution, and percentage calculation.

```
Terminal screenshot: npx hardhat test

> alu-digital-asset-protection@1.0.0 test
> hardhat test

ALUAssetRegistry and ALULogoToken
  ALUAssetRegistry ERC-721 registration
    ✓ registers the ALU logo successfully and returns token ID 1 (526ms)
    ✓ rejects a duplicate registration using the same hash
    ✓ verifyLogoIntegrity returns true for the correct logo hash
    ✓ verifyLogoIntegrity returns false for an incorrect logo hash
    ✓ getAsset returns the correct asset name and file type
  ALULogoToken ERC-20 tokenisation
    ✓ mints the full supply of 1,000,000 ALUT to the logo owner
    ✓ distributeShares transfers the specified tokens to a recipient
    ✓ ownershipPercentage returns the correct whole-number percentage

8 passing (711ms)
```

Figure 4: All eight Hardhat automated tests passing.

Conclusion

This assignment helped me understand how cryptocurrency platforms onboard users and how blockchain can protect digital assets beyond simple buying and selling. I learned how a file hash can prove whether a digital file is authentic, how ERC-721 can register a unique asset, and how ERC-20 can represent fractional ownership shares. If I had more time, I would add a web dApp for uploading a logo, automatically hashing it in the browser, verifying the hash against the contract, and displaying token ownership percentages in a simple interface.

GitHub Link and Video

GitHub repository: PASTE_YOUR_PUBLIC_GITHUB_LINK_HERE

Demo video link: PASTE_YOUR_VIDEO_LINK_HERE

References

Coinbase. Identity verification and KYC guidance. Coinbase Help Center.

Coinbase. Pricing and fees disclosures. Coinbase Help Center.

Hardhat. Hardhat Network and Getting Started documentation. Nomic Foundation.

OpenZeppelin. Contracts documentation for ERC-20, ERC-721, and Ownable.

African Leadership University. Official ALU website logo image and institutional information.